⚠ NUTZUNGSBEDINGUNGEN ⚠

Dieses Dokument dient als Lernhilfe für Klausuren. Wir übernehmen keine Gewähr für die Vollständigkeit und Richtigkeit der Inhalte. Mit der Nutzung dieses Dokuments erkennst du diesen Haftungsausschluss an.

Sollten dir Fehler auffallen oder etwas fehlen, schicke bitte eine private Nachricht an uns.

Viel Spaß beim üben,

Ömer und ***Luiz***

1. Informatik Klausur 16.12.2025

Fachbegriffe

Klasse

Eine **Klasse** ist ein **Bauplan zur Abbildung von Objekten**. Sie legt fest, **welche Attribute** Objekte besitzen und **welche Methoden** sie ausführen können. Klassen können von anderen Klassen **erben**.
(In Python class Klasse():)

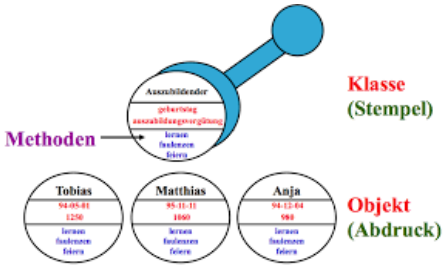
Objekt

Ein Objekt ist eine **konkrete Einheit**, die aus **einer Klasse erzeugt** wird (**Instanz** einer Klasse).

Methode

Methoden sind **Funktionen**, die innerhalb einer Klasse definiert sind und das **Verhalten der Objekte** dieser Klasse beschreiben.

(In Python "def Methode():")



Attribute

Attribute sind die **Eigenschaften der zu konstruierenden Objekte** (z. B. Name, Alter, Farbe).

Instanz

Eine Instanz ist die konkrete Realisierung einer Klasse, also ein erzeugtes Objekt.

Beziehungen

Beziehungen beschreiben die Art und Weise, wie Klassen und Objekte miteinander verbunden sind (z. B. Vererbung, Assoziation).

Kindklassen/Elternklassen

Vererbung ermöglicht es, eine neue Klasse (Kindklasse) zu erstellen, die die Attribute und Methoden einer bestehenden Klasse (Elternklasse) übernimmt und erweitern oder verändern kann.
(In Python: class Kindklasse(Elternklasse))

Konstruktor:

Ein Konstruktor ist eine spezielle Methode einer Klasse, die automatisch aufgerufen wird, wenn ein neues Objekt erstellt wird.
(In Python: __init__())

Kohäsion:

Kohäsion beschreibt den Grad des inhaltlichen Zusammenhalts einer Programmeinheit.
Hohe Kohäsion liegt vor, wenn eine Einheit genau eine klar abgegrenzte Aufgabe erfüllt.

Kopplung:

Kopplung ist das Maß dafür, wie stark **Klassen**, **Module** oder **Methoden miteinander verbunden sind** und sich **gegenseitig beeinflussen**. Je **geringer** die **Kopplung**, desto **unabhängiger** sind die Programmeinheiten voneinander.

Grundelemente der OOP

Vererbung
Vererbung ermöglicht es, eine neue Klasse auf Basis einer bestehenden Klasse zu erstellen. Die **neue Klasse (Elternklasse) übernimmt Attribute und Methoden** der **bestehenden Klasse (Kindklasse)** und kann sie erweitern oder verändern.
(In Python: class Kindklasse(Elternklasse))

Datenkapselung
Datenkapselung bedeutet das **Verbergen von internen Daten** eines Objekts vor direktem Zugriff von außen. Der Zugriff erfolgt kontrolliert über **Methoden** (z. B. Getter und Setter).
(in Python: __gekapselteVariable)

Polymorphie
Polymorphie bedeutet, dass **die gleiche Methode** bei **verschiedenen Objekten unterschiedlich** ausgeführt werden kann.
(Beispiel: Die Methode GibFigur() funktioniert bei einem Sofa anders als bei einem Stuhl.)

Abstraktion (wir hatten dies nicht im Unterricht)
Abstraktion bedeutet, dass nur die wichtigen Eigenschaften und Funktionen eines Objekts gezeigt werden, während unwichtige Details verborgen bleiben.

Vor- und Nachteile der OOP

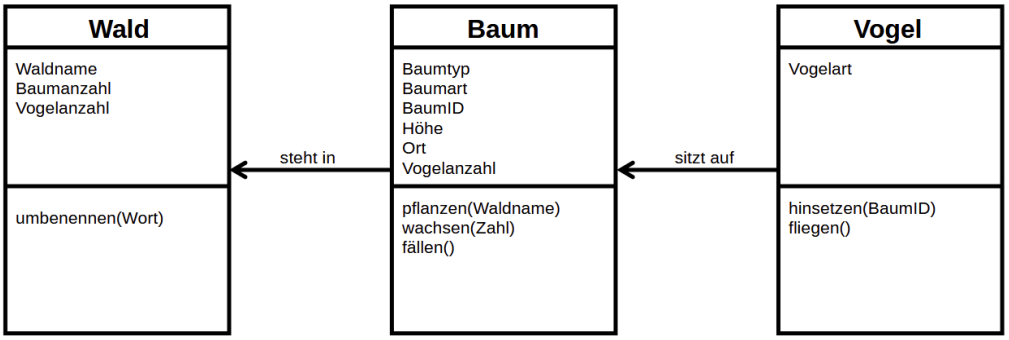
Vorteile	Nachteile
-Übersichtlich und nachvollziehbar -Modularisierung -Wiederverwendbarkeit durch Modularisierung -einfache Suche nach Fehlern durch Modularisierung -Geringer Programmieraufwand	-Teilweise intransparenter Code -Teilweise schwierig die Realität abzubilden

UML-Klassendiagramm

Unified Modeling Language Diagramme sind standardisierte Diagramme, die Softwaresysteme visuell darstellen, um diese zu dokumentieren und zu planen.

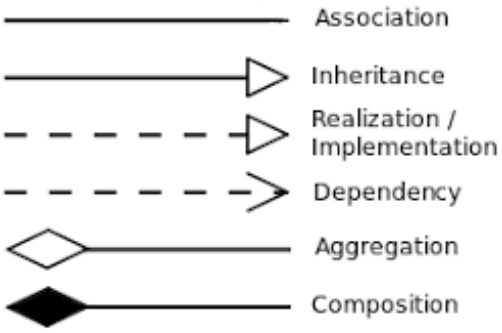
Klassenname
Attribute (Eigenschaften)
Methoden (Operationen, Befehle)

Es werden Beziehung zwischen den einzelnen Objekten, Instanzen einer Klasse, mit Pfeilen dargestellt.



Assoziation zeigt eine Verbindung an.
Bsp. Auto - Fahrer; Auto hat Fahrer, Fahrer hat Auto

Vererbung zeigt eine "ist ein" -Beziehung an, Kindklasse erbt von der Elternklasse.
Bsp. Tiger "ist eine " Großkatze, Tiger erbt Eigenschaften von Großkatze.



Abhängigkeit zeigt Abhängigkeit an.
Bsp. Auto ist auf Benzin angewiesen

Aggregation zeigt eine "hat"-Beziehung an
Bsp. Auto hat Räder

Komposition gibt eine "ist Teil von" Beziehung an.
Bsp. Buch hat Buchseiten, Buchseiten gibt es nicht ohne Buch.

grundlegende Python Befehle

Syntax

-Python-Code benötigt keine speziellen Start- oder End-Tags
-Einrückung (Indentation) ist sehr wichtig (meist 4 Leerzeichen oder 1 Tabulator)
-Kein Semikolon (;) nötig

```
print("Hallo Welt")
```

Variablen

Eine Variable ist ein Behälter zum Speichern von Daten

Python ist dynamisch typisiert, d. h. Der Datentyp muss nicht angegeben werden.

```
name = "Ömer"
nummer = 25
groesse = 1.80
ist_schueler = True
```

Operatoren

Operatoren sind Zeichen, mit denen man Rechnungen macht, Vergleiche anstellt oder Bedingungen verknüpft.

Operator	Name	Beispiel
=,	Zuweisungsoperatoren	name = „Ömer“ name → „Ömer“ nummer += 2 → nummer + 2
+,-,*,/,**,//	Arithmetische Operatoren	6*2 → 12
%	Modulo (gibt den Rest von Division)	5%2 → 1
==, <, <=, >, >=, !=	Vergleichsoperatoren	5 == 7 → False 9 == 9 →True 4 < 9 → True 5 >= 5 → True 2 != 3 → True
and	und	False and False → False True and False → False True and True → True
or	oder	False or False → False True || False → True True || True → True
not	nicht	not False → True not True → False

If, elif, else

If-Statement prüft, ob ein Bedingung wahr (True) ist und führt dann den Code im Block aus

Elif-Statement wird benutzt wenn es mehrere Bedingungen gibt

Else-Statement wird benutzt wenn keine der Bedingungen wahr (True) sind

```
if Bedingung:
    #Code, der ausgeführt wird
```

```
if wetter == "regnerisch":
    print("Nimm einen Regenschirm!")
elif wetter == "kalt":
    print("Zieh eine Jacke an!")
elif wetter == "sonnig":
    print("Sonnenbrille nicht vergessen!")
else:
    print("Unbekanntes Wetter...")
```

For-Schleife

Ein for-Schleife führt etwas mehrmals aus

range(n) erzeugt Zahlen von 0 bis n-1

```
for i in range(5):
    print(i)
```

→ 0,1,2,3,4

Methoden

def definiert eine Funktion (Methode)
Funktionen können Parameter haben

```
def begruessung(name):
    print("Hallo", name)
begruessung("Ömer")
```

→ Hallo Ömer

Wichtig!:

Methoden in Klassen starten immer mit self, sonst weiß Python nicht, auf welches Objekt sie sich beziehen.

Klassen

```
class KundenKonto(Konto):
    def __init__(self, kontostand):
        self.__kontostand = kontostand  # privat

    def einzahlen(self, betrag):
        self.__kontostand += betrag

    def get_kontostand(self):
        return self.__kontostand
```

__init__(self) → Konstruktor, wird automatisch aufgerufen, wenn ein Objekt erstellt wird

class KundenKonto(Konto) → KundenKonto ist Kindklasse, Konto ist Elternklasse

self.__kontostand ist **gekapselt**, es kann nur von der Klasse selbst, oder mit get_kontostand(self) erreicht werden